THE POST-COURSE CHALLENGE COLLECTION

Five Java Missions

Small toolkit. Serious programs.

Five original projects for someone who has finished *Initiation à la programmation en Java*.

01	**Touchdown** Pilot a lander; make a computer find a landing plan.	Guided
02	**Dead Drop** Build a desk for decoding scrambled transmissions.	Structured
03	**Prism Vault** Trace laser paths and repair a mirror puzzle.	Light guidance
04	**Clockwork** Build a tiny computer, its editor, and its debugger.	Specification only
05	**Blackout** Design a complete station-rescue game with undo and replay.	Independent

The promise Every required feature can be built with the book's Java concepts. The difficulty comes from combining them, designing representations, and getting boundary cases right.

English edition • Console programs • No solutions included

Based on the supplied 66-page EPFL BOOC by Jamila Sam, Jean-Cédric Chappelier, and Vincent Lepetit (EPFL Press, 2016). Original problem statements; not an official EPFL assessment.

BEFORE YOU START

The rules of the workshop

Your entire toolkit

Use Java console input/output, `Scanner`, `int`, `double`, `char`, `boolean`, constants, expressions, conditions, and loops. Use fixed arrays (including multidimensional and unequal-length rows), `String`, `ArrayList`, and your own `static` methods, including overloads.

The relevant library operations are the ones shown in the BOOC: `nextInt`, `nextDouble`, `nextLine`; `String.length`, `charAt`, `indexOf(char)`, `substring`, `replace(char,char)`, `equals`, `intern`; and `ArrayList.size`, `get`, `add`, `set`, `isEmpty`, `clear`, `remove(index)`. Mission 1 also uses `Math.sin` and `Math.PI`.

One class containing `main` and static methods per project is enough. Arrays and parallel lists can represent related data. You do not need custom object types, inheritance, recursion, maps, streams, regular expressions, sorting libraries, exception handling, graphical interfaces, networking, or file input/output. Keep all saved sessions in memory; hard-code the supplied fixtures. Build searches and copies with loops.

Input contract: challenging without untaught APIs

When a prompt requests a number, you may assume the user enters a finite number of the requested Java type that fits that type. You must still reject values outside the stated range. Text can be empty or invalid and must be checked. Do not require recovery from letters typed at an integer prompt or from end-of-input. Use separate prompts for commands and arguments; no general command parser is required. Handle the pending newline when mixing number reads and `nextLine` (BOOC p. 11).

What counts as finishing a mission?

1. A working console program implementing *all* required behavior, with useful prompts and readable output. All sections are required unless explicitly marked optional.
2. A short test log: input or starting state, expected result, actual result. Predict results before running the program. Plain Java test methods that print PASS/FAIL are enough.
3. A brief design note: your data representation, important method contracts, and the explanation requested at the end of the mission.

Keep calculations and state changes separate from keyboard prompts wherever practical. Tests should call the same logic as the interactive program. Do not replace the hard part with a library call.

Progression and pace

These are projects, not short syntax drills. Expect the later ones to take several sittings; no speed target is intended. Do them in order. Guidance fades, but the rules and acceptance criteria stay explicit. The final project reuses concepts from the book's game study without requiring you to finish or copy Puissance 4.

A useful discipline Before writing a substantial method, state its inputs, result, and exactly which data it may change. Before fixing a bug, write down a case that exposes it.

MISSION 01 / Guided / numerical simulation

Touchdown

A lunar lander is descending. You control the thruster once per second. Then you must teach your program to search for a safe landing plan.

The flight model

The state is altitude h in metres, vertical velocity v in metres per second (positive means upward), whole fuel units f, and completed turns. Default start: $h = 60.0$, $v = -4.0$, $f = 24$, turn 0. For custom starts accept $0 < h \leq 100$, $-10 \leq v \leq 0$, and $0 \leq f \leq 80$.

On turn t, starting at $t = 1$, choose an integer thrust $u \in \{0, 1, 2, 3, 4\}$ with $u \leq f$. A valid turn performs these updates *in this order*:

$$f \leftarrow f - u, \qquad v \leftarrow v - g(t) + u, \qquad h \leftarrow h + v.$$

There are two environments, chosen before a flight:

$$g(t) = \begin{cases} 1.6 & \text{calm,} \\ 1.6 + 0.2\sin(t\pi/6) & \text{pulsing gravity.} \end{cases}$$

This is a deliberately discrete toy model. Use the updated velocity for altitude; do not interpolate the instant of contact with the ground.

After the update, if $h \leq 0$, the flight ends: **safe** if $-3.0 \leq v \leq 0$, otherwise **crashed**. If still airborne after turn 30, it ends as **timed out**. A landing on turn 30 takes priority over timeout. Zero fuel still permits $u = 0$. Invalid thrust consumes neither fuel nor a turn.

Build it in four passes

1. **One turn.** Implement and test the arithmetic on known states before introducing a keyboard loop. Use `double` for h, v, and gravity; use `int` for fuel, thrust, and turns.
2. **Manual flight.** Print the state after every valid turn, then a final outcome, fuel used, and contact velocity if applicable. Let the player retry the same initial scenario without restarting the program.
3. **Automatic trials.** Simulate a proposed thrust policy without asking questions or printing a line per turn. Each trial must begin with fresh state.
4. **Flight planner.** Search the policy family on the next page and replay the selected policy as an ordinary flight trace. Support both environments and custom starting states.

Suggested method boundaries Try separate methods for gravity, thrust validation, one-step calculations, trial simulation, and reporting. First write their headers and contracts. Decide how a trial returns the few results the planner needs.

MISSION 01 / CONTINUED

A pilot that searches

The policy family

Choose three integers: $a, b \in \{0, 1, 2, 3, 4\}$ and $s \in \{0, \ldots, 30\}$. Use thrust a on turns 1 through s, then thrust b on all later turns. For $s = 0$, use b immediately. For $s = 30$, a flight never reaches the second phase.

Try every triple (a, b, s) with loops. Stop a trial at touchdown, timeout, or the first unaffordable thrust. Unaffordable thrust makes that trial invalid; do not silently reduce it. Compare safe trials by:

1. least fuel used;
2. then smallest impact speed $-v$;
3. then fewest turns;
4. then smallest a, then smallest b, then smallest s.

When comparing impact speeds, differences of at most 10^{-9} count as ties. Enumerate triples in increasing a, then b, then s order for reproducibility. Report `NO SAFE POLICY IN THIS FAMILY` when none works. That does not prove no possible sequence of thrusts could work.

Implement two meaningful overloads: `gravity(int turn)` for calm mode, and `gravity(int turn, boolean pulsing)` for either mode. Make the first reuse the second. Store fixed settings such as the turn limit as named constants.

Checks you can trust

Setup / action	Expected result
Calm: $h = 6$, $v = -2$, $f = 6$; thrusts $1, 2, 2$	After turn 1: $(h, v, f) = (3.4, -2.6, 5)$; turn 2: $(1.2, -2.2, 3)$; turn 3: $(-0.6, -1.8, 1)$, safe.
Same start; thrusts $0, 1$	Crash on turn 2: $h = -1.8$, $v = -4.2$, fuel 5.
Fuel 2; request thrust 3	Reject it; every part of the state is unchanged.
Default calm scenario	The search must find at least one safe policy. Discover which one yourself.
Default altitude/velocity, fuel 0	No safe policy in this family, in calm mode.
Pulsing mode, turn 3	$g(3) = 1.8$ (within floating-point tolerance).

Use a tolerance of 10^{-9} when checking displayed numerical examples. Test contact velocity exactly -3.0, a turn-30 landing, a timeout, $s = 0$, and $s = 30$ separately.

Explain before moving on

Why is using the old velocity a different model? Why is `1/2` different from `1/2.0`? Identify one local variable, one constant, and one boolean condition in your code. Show that changing a numeric parameter inside a method does not update the caller's variable. Distinguish a compilation error, an execution failure, and a logically wrong flight result.

MISSION 02 / Structured / strings and data

Dead Drop

You intercept messages from a courier network. Each transmission has been shifted and scrambled. Build a tool that can write them, recover them, and explain competing interpretations.

The transmission format

Messages contain only uppercase letters `A`–`Z`, ordinary spaces, and the punctuation `. , ! ?`. Empty messages are valid. Reject any other character. Preserve every space and punctuation mark, including spaces at the start and end.

Encryption has a key $k \in \{0, \ldots, 25\}$ and a block width $w \in \{2, \ldots, 9\}$:

1. Shift each letter forward by k positions in `ABCDEFGHIJKLMNOPQRSTUVWXYZ`, wrapping at the end. Other characters are unchanged.
2. Cut the resulting string into consecutive blocks of w characters, counting *all* characters. Reverse each block independently. Reverse the final shorter block too.

Decryption must undo these transformations in the correct order. The alphabet string and `indexOf(char)` let you work with letters without assuming character-code arithmetic or casts.

Your analysis desk must offer

- **Encode / decode:** enter a message, key, and width; display the result inside delimiters so spaces remain visible. Decoding accepts the same character set as encoding.
- **Intercept analysis:** enter ciphertext and an optional crib (a phrase you believe occurs in the plaintext). Try all 208 combinations of width and key. An empty crib imposes no filter; otherwise keep only candidates containing that exact contiguous substring, with case and spacing unchanged.
- **Candidate ranking:** score each remaining plaintext with the dictionary below. A word is a maximal nonempty sequence of `A`–`Z`; all other characters separate words. Add one point for *each occurrence* of a dictionary word. Partial matches do not count.
- **Evidence:** show the three highest-ranked parameter pairs (or all if fewer), their plaintexts and scores, the total number of survivors, and how many share the maximum score. Retain separate pairs even if their plaintexts are identical. With no candidates, report that explicitly.

```
MEET AT THE OLD DOCK NINE BRING KEY NORTH SOUTH EAST WEST
```

Use that exact 12-word dictionary. Rank by descending score, then ascending width, then ascending key. Implement the ranking yourself with scans and comparisons; no sorting API. A score is evidence, not proof that a message is correct.

MISSION 02 / CONTINUED

Make the evidence inspectable

Beyond a single decoded line

For a selected candidate, print a 26-element letter-frequency table and a masked preview in which all occurrences of one user-selected uppercase letter are replaced by `*`. The original candidate must remain unchanged. Also allow an excerpt with start index included and end index excluded; accept $0 \leq \text{start} \leq \text{end} \leq \text{length}$. An empty excerpt is valid.

Provide a mode that shows every winning candidate when the maximum score is tied. The user must be able to inspect ambiguity instead of having the first trial silently presented as the answer. Frequency counts ignore spaces and punctuation.

A few boundaries, not a recipe

Keep encryption, decryption, substring matching, and dictionary scoring callable without keyboard input. Fixed arrays are sufficient for the candidate data. Decide their layout and how to represent the number of valid entries. Use at least one enhanced `for` loop where an index is unnecessary, and an indexed loop where it is necessary.

You may find these two contracts useful:

```
static String encode(String message, int key, int width)
static int score(String plaintext, String[] dictionary)
```

The arguments to `encode` may be assumed valid after the input layer has checked them. Neither method should print or alter its inputs. Design the other method contracts yourself.

Acceptance cases

In this table, vertical bars mark the boundaries of a string; they are not part of the data.

Operation	Expected result
Encode `|AB Z!|`, $k = 1, w = 3$	`| CB!A|`; note the leading space.
Encode `|ATTACK AT DAWN!|`, $k = 3, w = 4$	`|DWWDD NFDG W!QZ|`.
Decode either example	Recover the original character for character.
Score `|MEET AT THE OLD DOCK AT NINE.|`	7; the second `AT` counts again.
Score `|THEATER,THE!|`	1, not 2.
Analyse empty ciphertext, no crib	208 candidates, all score 0; first three pairs (w, k) are $(2, 0), (2, 1), (2, 2)$.
Excerpt $[0, 0)$; absent crib	Empty excerpt; zero candidates if no decoded text contains the crib.

For every key and width, test that decoding an encoded valid message recovers it exactly. Include length 0, 1, $w - 1$, w, and $w + 1$, repeated separators, and a word ending at the last character. For each candidate, the sum of its histogram must equal its number of letters.

Explain before moving on

Why must typed commands use `equals`, not `==`? Demonstrate that `replace` does not modify its source string. Predict and then observe how `intern()` affects reference comparison with an identical literal; keep `equals` in your actual command logic.

MISSION 03 / Light guidance / grids and exhaustive trials

Prism Vault

A vault opens only when its laser touches every target. Your tool must trace the beam, detect traps, and find the smallest mirror repair that opens the vault.

World rules

Use a rectangular grid between 1×1 and 12×12. A cell is empty `.`, a wall `#`, a target `T`, or a mirror `/` or `\`. There are at most 12 mirrors. The starting row, column, and direction are separate data; the starting cell must be empty. Directions are north, east, south, west; north decreases the row and east increases the column. All coordinates start at zero.

The first beam state is the starting cell and direction. On every iteration:

1. If outside the grid, finish as **escaped**.
2. If this exact (row, column, incoming direction) has appeared before, finish as **looped**, without recording the repeated state again.
3. Record the state. A wall finishes the trace as **absorbed**. A target is lit and the beam passes through. Count each target only once.
4. A mirror changes the direction according to the table; other traversable cells keep it. Move one cell in that direction.

Incoming	North	East	South	West
`/` sends you	East	North	West	South
`\` sends you	West	South	East	North

The trace length is the number of recorded states, including an absorbing wall. Leaving the grid adds no state. A repeated cell with a *different* direction is not automatically a loop. A grid can record at most $4RC$ states before stopping, where R, C are its dimensions.

Required laboratory

Load a grid from prompted row strings and validate dimensions, symbols, and the start. Display the grid with coordinates. Offer trace, rotate one mirror, reset to the original grid, and automatic repair. Rotation exchanges `/` and `\`; rotating any other cell is rejected without change. A trace reports its ordered states, lit targets, length, and termination reason. It must never modify the puzzle grid.

Keep an independent original grid for reset. Show whether the current grid has the same *contents* as the original, even if it is a different array. You choose the representation of trace history and target visits.

MISSION 03 / CONTINUED

Repair the vault

The repair search

A repair rotates zero, one, or two *distinct* existing mirrors exactly once. All trials begin from the current grid. A successful trial lights every target; its final termination reason may be escape, absorption, or a loop. Thus a beam can open the vault and then loop forever physically; your simulation still terminates.

Select a successful repair using the following order:

1. fewest rotated mirrors;
2. then shortest trace;
3. then earliest coordinate list: sort each repair's coordinates by row, then column, and compare the first differing coordinate in that same order.

For zero targets, every trace lights all targets, so the empty repair wins. If no repair works, report `NO REPAIR WITH AT MOST TWO ROTATIONS`. The search must leave the current grid unchanged. Provide a separate action to apply the recommended rotations; tracing the applied result must reproduce the prediction.

Do not use recursion or a general pathfinding algorithm. There are only finitely many choices of zero, one, or two mirrors; the challenge is to enumerate them without duplicates and isolate each trial's state.

Small fixtures that expose large bugs

Grid and start	Expected trace result
One row `.T.`; start $(0, 0)$ east	Escaped, length 3, one target lit.
One row `.#T`; start $(0, 0)$ east	Absorbed, length 2, zero targets lit.
One row `./T`; start $(0, 0)$ east	Escaped, length 2, zero targets lit. Rotating this mirror still does not reach the target.
Loop fixture below; start $(0, 1)$ east	Looped, length 8, zero targets. The repeated starting state is not counted twice.

```
/.\
...
\./
```

Create and document at least four further puzzles: one already open, one requiring exactly one rotation, one requiring exactly two, and one impossible under the repair budget. Verify minimality by inspecting the search results. Include a case where a target is visited more than once and one where the same cell is entered from different directions.

Evidence of understanding

Show a test in which you copy a grid, modify the copy, and prove that every original row remains unchanged. Explain why copying only the outer array fails. Show two grids whose contents match but whose references differ. Explain why a cell-only visited flag can stop a valid trace too early, and why trying one repair must not contaminate the next.

MISSION 04 / Specification only / a programmable machine

Clockwork

Build a miniature computer that runs programs you can edit while the application is open. Give it a debugger that can step backward as reliably as it steps forward.

The machine contract

There are eight tape cells, initially zero, each holding an integer from 0 to 9. The tape pointer starts at cell 0. The instruction pointer starts at line 0. Output is initially empty. Program lines are numbered from zero. Instructions and their semantics are:

Instruction	Effect of one step
`ADD n`	Add n to the current cell, wrapping into $0 \dots 9$; accept $-9 \le n \le 9$. Then advance one line.
`MOVE n`	Move the tape pointer by n, where $-7 \le n \le 7$. Fault if the destination is outside the tape; otherwise advance one line.
`PRINT`	Append the current digit to output, then advance one line.
`JZ p`	If the current cell is zero, jump to line p; otherwise advance one line. Validate p against the current program only when the jump is taken.
`JMP p`	Jump to line p. Fault if it is outside the current program.
`HALT`	Mark the machine halted; keep the instruction pointer on this line.

At entry, and after a successful instruction, reaching exactly one line beyond the last line means normal halt. The empty program is therefore halted immediately. A fault changes nothing in the machine and creates no undo entry; report its line and reason. It does not become a successfully executed instruction.

Jump operands may be any nonnegative `int` when edited. All other operand ranges above are checked during editing. This permits unfinished programs to contain forward jumps. `PRINT` and `HALT` have no user-supplied operand.

The editor contract

Use `ArrayList` for the program, including an appropriate use of `ArrayList<Integer>`. Let the user append a line, replace a line, delete a line, list the program, or clear it. Invalid edits leave everything untouched. Deleting a line renumbers later lines; *numeric jump operands do not change*. Make this visible in the interface. An accepted edit resets execution to fresh tape, pointer, output, and undo history. A cleared program is immediately halted.

Enter the opcode and operand through separate prompts. Compare opcode strings by content. Choose your own data representation and method boundaries. Your program must remain usable after empty input, invalid opcodes, invalid indices, and invalid operand ranges.

MISSION 04 / CONTINUED

The debugger is part of the machine

Execution controls

Implement **step**, **run**, **undo**, **reset execution**, and **inspect**. Run executes until halt, a fault, or 1,000 successful instructions during that run request. The limit pauses execution without resetting it; another run may continue. If the 1,000th instruction halts, report halt rather than pause. A step on an already halted machine is a no-op.

Undo restores the state immediately before the most recent successful instruction, including every tape cell, both pointers, output, and halted status. It works after `HALT` and after an instruction that falls off the end. Multiple undos are required; no undo is available before the first instruction. A run creates one undo point *per instruction*. A fault creates none. There is no redo command.

Inspect shows tape contents, the current cell, the current program line or end position, output, status, and available undo count. Reset execution keeps the program but discards the entire execution history. Use the book's dynamic-list operations where appropriate; you should have real uses for adding, replacing, removing, clearing, checking emptiness, and reading by index.

Deliver this demo program

```
0  ADD 3
1  PRINT
2  ADD -1
3  JZ 5
4  JMP 1
5  HALT
```

From a fresh machine this prints `321`, halts on line 5 after 13 successful instructions, and leaves cell 0 equal to zero. Undoing the halt makes the machine runnable on line 5 again, with output still `321`. Demonstrate the program both by stepping and by running.

Required evidence

- Test wraparound in both directions, a move beyond each end, a bad taken jump, a bad untaken `JZ`, falling off the program, and the empty program.
- A program consisting only of `JMP 0` pauses after 1,000 instructions and has 1,000 undo points. Undoing one restores its preceding state even though its visible tape is identical.
- Execute several instructions, undo them all, and compare every state component with the initial state. Re-execute and recover the same output. Include an edit after partial execution.
- Create two more programs of your own: one that visits multiple cells and prints at least six digits, and one that uses a zero test to take different paths from two explicitly prepared tape states. Include predicted traces.

Explain the design

Which data belongs to the program and which belongs to an execution? Why does an array stored in a history list still need an independent copy? Demonstrate the difference between changing an array element in a method and assigning a new array to that method's parameter. Explain your handling of negative wraparound.

MISSION 05 / Independent / comprehensive capstone

Blackout

An orbital station is burning. You have limited oxygen, a water tank, locked doors, and people to rescue. Build a complete, deterministic game that lets the player rewind mistakes and replay a mission.

The world contract

Each mission is a rectangular map of at most 12 × 16 cells. Use these symbols for authored maps:

`#`	Wall	`.`	Floor
`S`	One player start	`E`	One exit
`D`	Closed door	`K`	One key pickup
`C`	One survivor	`O`	Oxygen pickup
`W`	Water pickup	`F`	Initial fire on floor

A mission must have exactly one start and one exit, and at least one survivor. Treat `S` as floor after placing the player. An opened door becomes floor. Initial fire is separate from the underlying floor. Fire may coexist with the player or exit; it destroys pickups and survivors as specified on the next page. A cell contains at most one original map symbol.

Start every mission with oxygen 20, water 3, keys 0, no rescued survivors, and turn 0. Oxygen is capped at 20 and water at 5. The total number of survivors is fixed from the initial map. All movement and adjacency use north, east, south, and west, never diagonals. Coordinates start at zero.

Player actions

Action	Effect before the environment advances
Move	Move one cell in a chosen direction. Outside the map, walls, and closed doors are invalid destinations. Burning cells are legal destinations. On entry, collect a pickup or rescue a survivor automatically. A key adds 1; oxygen adds 6 up to its cap; water adds 2 up to its cap. Collected cells become floor.
Open	Choose an adjacent closed door. With at least one key, spend one key and turn the door into floor. Otherwise reject the action. You do not move through it yet.
Spray	Choose an adjacent burning cell. With at least one water unit, spend one and extinguish that cell. Otherwise reject the action. You cannot spray your own cell.
Wait	Stay in place. This is always a valid action while the mission is active.

Each accepted action consumes exactly one turn. Reject an invalid action with an explanation and *no state change*. Menus, inspection, undo, reset, and replay never consume turns. A direction may be entered through a separate prompt.

MISSION 05 / CONTINUED

One turn has one meaning

Environment and outcome order

Resolve each accepted action in precisely this order:

1. Apply its immediate effect, including movement, collection, opening, or spraying.
2. Increase the turn number by one and decrease oxygen by one.
3. If the new turn number is divisible by three, spread fire. Every cell burning at the *start of this spread phase* ignites its adjacent cells, except walls and still-closed doors. Existing fire persists. Newly ignited cells do not spread again in the same turn.
4. Any pickup on a burning cell is destroyed; any survivor on a burning cell is lost. A rescued survivor is already safe and cannot be lost later. These effects leave floor beneath the fire. The exit remains an exit.
5. If the player's cell is burning, decrease oxygen by two more. Do this on every accepted turn, not just spread turns. Clamp negative oxygen to zero.
6. If oxygen is zero or any survivor has been lost, the mission is **lost**. Otherwise, if all survivors have been rescued and the player is at the exit, the mission is **won**. Otherwise it continues. Loss takes priority over victory.

Extinguishing a cell before a spread phase removes it as a source for that phase; a different burning neighbour can immediately ignite it again. Opening a door before that phase allows fire to enter the newly opened cell. A pickup collected before the oxygen cost can save the player. These are gameplay rules, not implementation suggestions.

A complete playable product

The console must show a coordinate-labelled map, player position, fire, resources, turn, rescue progress, and legal command names. When several things occupy a cell, choose a clear display convention and include a legend. Always make it possible to tell whether the player or exit is burning. Winning and losing stop normal actions, but still permit inspection, undo, reset, replay, and mission selection.

Provide unlimited undo within the current in-memory session: one undo restores the state before the last accepted action, including doors, pickups, fire, resources, position, rescued/lost counts, turn, and outcome. After undo, a new action creates a new continuation; abandoned actions must not reappear in replay. Invalid actions and administrative commands never enter the action history.

Replay runs the retained accepted actions from the mission's original state and displays the resulting sequence. It must reproduce the current complete state, whether active, won, or lost. After viewing replay, the player's current game and undo history must remain intact. Reset returns to the authored mission and clears its current history.

You own the architecture Choose your representations, method contracts, decomposition, and development sequence. There is deliberately no list of helper methods or suggested engine structure. Deliver a short design document explaining those choices and how you know they work.

MISSION 05 / CONTINUED

Build a game worth replaying

The supplied calibration mission

This 7×11 map follows the rules above. Spaces shown outside the box are not map data.

```
###########
#S..K.D.CE#
#.###.###.#
#O..#...W.#
###.#.###.#
#F..#.....#
###########
```

It is winnable from the standard starting resources. Find and retain a successful command transcript; no solution route is provided. After one *wait* from the start, turn is 1, oxygen is 19, and fire is unchanged. After three consecutive waits, oxygen is 17 and the fire occupies exactly $(5, 1)$ and $(5, 2)$. This checks that one spread cannot chain across an entire corridor.

Your release must include three missions

Keep this calibration mission and author two additional ones. At least one authored mission must contain two or more survivors, and at least one must make extinguishing fire necessary for a win. Both authored missions must include a meaningful timing or resource choice. Keep the maps fixed in the source, provide titles and a short briefing, and allow selection without restarting the program. Selecting a mission starts it fresh and discards the previous current session.

Use a two-dimensional array whose rows can have different lengths to store each mission's best successful command sequence for this application run. Initially each row is empty. The first win sets the record; replace it only when a shorter winning sequence is achieved. Encode actions as integers with a documented convention. Replaying a stored best sequence is a separate viewing action that does not replace the current game. Best records survive mission selection and reset but need not survive closing the application. Undo does not erase an already achieved best record.

For each authored mission, provide a winning transcript and explain the intended decision. For the mission requiring spray, explain why no winning no-spray route exists; a deliberately forced corridor can make this argument small and inspectable. You do not need an automatic solver or a random map generator.

Release evidence

- Verify each action's valid and invalid cases, pickup caps, keys being spent, a closed door blocking fire, and an opened door admitting it.
- Test simultaneous spread, spray followed by re-ignition, a survivor burning, oxygen reaching zero at the exit, and rescue of the last survivor before a later fire phase.
- Show that inspecting and replaying never change the live game. Undo a pickup, a door opening, a spread turn, a win, and a loss; compare *all* state components.
- Show an undo followed by a different action and a replay of that new history. Test reset, switching missions, and replacement of a best record by a shorter win.

Final deliverables: the playable program, three briefings, the design document, a test log with at least 15 deliberate cases, and three winning transcripts. A beautiful terminal display is welcome, but correct rules and worthwhile decisions come first.

AFTER THE FIVE MISSIONS

What you have actually tested

This is an audit of the five projects, not a sixth exercise. Page numbers below refer to the supplied BOOC; its printed numbering matches the PDF pages.

Course concepts	BOOC	Where the understanding becomes visible
Compilation, execution, debugging	Lesson 1 pp. 5–6	M1 error explanations; every project compiles and is checked against predictions.
Variables, types, assignment, constants, input/output	Lessons 2–3 pp. 7–11	M1 flight state and menus; M2 mixed text/number input; M4 machine state.
Expressions, integer division, remainder, mathematical functions	Lesson 4 pp. 12–13	M1 gravity and numerical model; M2 letter rotation; M4 digit wraparound.
Branches, comparisons, logic, booleans	Lessons 5–7 pp. 14–20	M1 outcomes; M3 trace termination; M5 validity and loss/win precedence.
`for`, nested loops, `while`, `do...while`, scope	Lessons 8–12 pp. 21–28	M1 policy enumeration and retry prompts; M2 scans; M3 traces; M5 game loop. Use each loop form appropriately at least once across the collection.
Array construction, defaults, indexing, traversal, enhanced `for`	Lessons 13–15 pp. 29–34	M2 candidate data and histogram; M4 tape initialization and bounds.
References, content comparison, independent copying	Lesson 16 pp. 35–36	M3 original versus trial grids; M4 undo; M5 replay isolation.
Multidimensional arrays and unequal row lengths	Lesson 17 pp. 37–38	M3 grids; M5 maps and differently sized best-action sequences.
`char`, `String`, concatenation, literal pool, text operations	Lessons 18–20 pp. 39–43	M2 cipher, cribs, token scoring, excerpts, masking, and the `intern` explanation.
`ArrayList`, wrapper types, dynamic operations	Lesson 21 p. 44	M4 editable program and history; M5 actions and undo. Use an `ArrayList<Integer>` somewhere appropriate and explain automatic boxing.
Method calls, parameters, return values, `void`, scope, contracts, overloads	Lessons 22–28 pp. 45–52	M1 gravity overloads; M1/M4 parameter demonstrations; decomposition throughout.
Incremental development, board display, valid moves, engine, termination	Lessons 29–35 pp. 53–65	M3 puzzle laboratory; M5 complete game and regression evidence.

The real finishing test Can you explain why your program is correct on its boundary cases, change a rule without breaking unrelated behavior, and reproduce a bug from a short transcript? If so, you have gone well beyond recognizing Java syntax.